

Homework 2 Summary

Logan Clevenger

CPSC – 224 – 02

9/30/2026

Goal/Purpose of Program:

The goal for this program is to expand on my previous implementation of *Farkle*. Building on the previous functionality, the player should be able to enter their name at the beginning of the game. From there, points are tied to this specific player, and while playing, they can also choose to quit the game or bank their current turn points. Additionally, dice rolling is no longer limited to a single group roll; the game now allows a player to continue rerolling dice not selected in their Meld. Also, the “Hot Hands” feature has been added, which allows for another complete 6-dice roll if a player is able to get all 6 of their current dice in play into a scoring meld.

Overview of General Program Design:

For this project, I’ve so far utilized 5 different classes. I first made a Die class that represents one die in the *Farkle* game. This class stores its current numeric value and is able to roll randomly to another value (between 1-6). I then made a Hand class that owns 6 die instances. This class also tracks selections being made between the player’s hand and their meld, along with whether or not the player rolled a Farkle. Following that, I made a Meld class. This class stores selected dice from the player’s hand and calculates the Zag Farkle score of the dice currently contained in the player’s meld. Next, I made the Farkle class. This class actually runs the game and controls menu display and user input for die selections. In homework 2, the Farkle class was further expanded to handle rerolling, banking player points, and recognizing/celebrating Hot Hands. I also have added the Player class. This class allows for storage of the current player name and the banked points attached to this player.

Programs Starting Screen With Intro Banner and First Meld Menu:

```
Please enter player name: Logan!

*****
*                               *
*       Zag Farkle by Logan Clevenger!       *
*                               *
*       Copyright: 2026                       *
*                               *
*****

*****Let's Play Farkle!*****
Player: Logan!
Hand          6      4      4      1      1      3
Quantity of each die value: 2      0      1      2      0      1
***** Current hand and meld *****
*****
Die   Hand   | Meld
-----+-----
(A)   6      |
(B)   4      |
(C)   4      |
(D)   1      |
(E)   1      |
(F)   3      |
-----+-----
Meld Score: 0

(Q) Quit game
(X) Confirm Meld

If entering multiple inputs - enter letters with no spaces!
Enter letter for your choice: A,B,C,D,E,F,X,Q: █
```

Unit Test Description:

One of the primary unit tests that I implemented and tested if my program passed was `testStraight()` in my `FarkleTest.java` file. This test is supposed to check that rolling a straight and selecting it into the meld would correspond to the correct amount of awarded points (1000) and also have the proper amount of scoring dice in the meld (6). In this test, I created a new `Meld` object and used the `.addDie()` method to pass in `Dice` objects with values 1-6 into the meld. From there, I used the method `.calculateScoreAndScoringDice()` on the meld and stored the results. Finally, I was able to use `assert` statements to check that the computed score and scoring dice were accurate for a straight. This demonstrates correct behavior as this displays my coded Farkle game following the scoring rules of Zag Farkle.

Major Design/Programming Challenges and Solutions:

1: One of my first major challenges was understanding how I could implement new functionality into my code without breaking what was written in my original program. I dealt with this by creating the testing file farkle2 (which I later deleted), but this allowed me to copy over code from Farkle that I had already written and edit it without fear that I would destroy my original functionality. From there, I just incrementally changed methods until I felt comfortable merging what I had in farkle2 back into my original Farkle file.

2: I also struggled with managing the game state across multiple rolls. It was hard to keep track of which dice had already been stored away in the meld, which dice were available for reroll, and what dice in the meld should be counted into the current turn score and how. To aid with this, I added helper methods like `.getAvailableValueTotals()` and `.resetDiceBackToHand()`, which allowed me to have more control over what was being counted in turn score and where dice were located.

Retrospective:

One of the things I would change looking at my project in retrospect was how I initially approached adding the new functionality to my game. I did not spend enough time planning how the new code/methods would connect to my current structure. So although I did create that editing file farkle2 and worked to preserve my code, I believe some of my solutions took me longer to come up with / act more clunkily than they would have if I had a better idea of the whole program.